

Artificial Intelligence (CS-203)

Objectives:

- **Student Program**
-

Solving Mathematical Problems

➤ Student Program

Student is one of the early language understanding program, written by Daniel Bobrow as his Ph.D. research project in 1964. It is designed to read and solve the kind of word problems found in high school algebra books. For example;

“if the number of customers Tom gets is twice the square of 20 % of the number of advertisements he runs, and the number of advertisements is 45 , then what is the number of customers Tom gets ?”

Student could correctly find that the number of customers is 162. To do this student must be far more sophisticated than Eliza; it must process and “understand” a great deal of the input, rather than just concentrate on a few key words. And it must compute the response rather than just fill in blanks. To transform the English paragraph it also employs the pattern-matching algorithm. But the translating algorithm is itself recursive process. Further solving of equations and isolating a variable is also a recursive.

➤ Example Calls

Student can easily solve the following descriptive example questions:

1. what is twice of square of z . if z is 3
2. The average score is 73 . the maximum score is 97 . what is the square of the difference between the average and maximum scores ?
3. Tom age is twice of Mary age . James age is half of the difference between Mary and Tom ages . Mary is 18 years old . how old is James?
4. The price of a radio is 680 rupees . if this price is 15 % less than the marked price . find the marked price .
5. x plus y is equal to z minus 5 . find square of z . 6 minus x is 4 . and 8 divided by y is $\frac{1}{4}$.

6. If the number of customers Tom gets is twice of the square of 20 % of the number of advertisements he runs . and the number of advertisements is 45 . if the profit Tom receives is 10 times the number of customers he gets . what is the profit ?
7. The cost per day of living for a group is the overhead cost plus the running cost for each person times the number of people in the group . This cost for one group equals \$ 100 . the number of people in the group is 40. If the overhead cost is 10 times the running cost . if running cost per person is 2 \$. find the overhead and running cost for each person .
8. Fran age divided by Robin height is one half of Kelly IQ . Kelly IQ minus 30 is Robin height. If Robin is 4 feet tall . how old is Fran ?
9. The cost of MCS is equal to admission fee plus tuition fee of all semesters. Tuition fee is equal to semester fee multiplied by total number of semesters of MCS program. Total number of semesters in mcs program is 4. Semester fee is 40000. Admission fee is 12000. Find the cost of mcs program.
10. 75 % attendance is required for every course. attendance can be calculated as sum of class attendance and lab attendance. percentage can be calculated as attendance of a student multiplied by 100 divided by total classes held. what is percentage of Asif. if his class attendance is 30 . lab attendance is 20. While total classes held are 60.
11. Sum of theory and practical marks obtained by a student makes the total marks of the student for a course. Similarly, Theory marks are calculated by adding Internal marks and marks obtained during exams. Moreover, marks of exams can be further distributed into Mid exam and Final Exam. Internal marks can be divided into Quizzes and Assignments. Find total marks obtained by Asif if he has obtained . 8 Marks in Quizzes. 6 marks in Assignments. 10 marks in Mid . 20 marks in Final exam. Finally, 12 in practical lab work.

➤ Student Modules (Code of Student)

There are two main modules of Student

1. First module to convert the English paragraph to mathematical expressions
2. Second module to solve the list of simultaneous linear equations

```
def student():
    userinput = input("Enter Statement:")
    userinput = userinput.lower()
    engEqs = userinput.split('.')
    eqs = []
    for engeq in engEqs:
        engeq = engeq.split()
        eqwlst = remove_noise(engeq)
        eqs.append(translate_to_exp(eqwlst))
    eqs = [eq for eq in eqs if eq != []]
    solve_equations(eqs)
```

as shown in the above definition of the student first module is named as **translate_to_expression** and the second module is termed as **solve_equations**. But before translating the words question noise words are being remove with the help of predefined **remove-if** function.

➤ Removing noise words

Noise-word-p is a single line user defined function shown below;

```
def remove_noise(txt):
    nwords = ['a', 'the', 'number', 'of', 'this', 'if', 'that', '$', 'similarly', 'finally', 'moreover', 'further', 'and', 'his', 'while']
    for w in nwords:
        if txt.count(w) > 0:
            txt = [a for a in txt if a != w]
    return txt
```

➤ Module 1. Translating English Paragraph to Mathematical Expressions

As Humans are slow in computing but excellent in pattern matching. Humans use this technique of problem solving extensively in their daily life. Similarly AI programs also use the pattern matching technique to solve the given problems. Similarly this module also use the pattern-matching algorithm (as used in Eliza) to translate the English paragraph into list of simultaneous linear equations. Input of this module is an English paragraph in the form of list of words from which noise words like “a” “an” etc. have already been removed.

Logic is very simple it parse/ break up the input paragraph into pieces and sub-pieces till it reach very simple small segments of sentences. These small segments are either numbers or variables if so then returned as it is, or they can be some very simple and directly translatable. Like when it is given “twice of something” or “half of something” will be directly converted to $[2 * ?x]$ and $[?x / 2]$ respectively. Note that the equivalents of English sentences are

mathematical expressions in Python syntax, where ?x can be replaced with anything (from a single number to a further complex expression). These direct conversion of small English sentences into equivalent mathematical expressions are done with the help of you can say a conversion table or in AI terminology with the help of set of student rules. Some rules are shown below;

```
( 'find ?*a' , [ 'To-find' , '=' , '?a' ] ) ,
( '?*a is equal to ?*b' , [ '?a' , '=' , '?b' ] ) ,
( '?*a equal to ?*b' , [ '?a' , '=' , '?b' ] ) ,
( '?*a is ?*b' , [ '?a' , '=' , '?b' ] ) ,
( 'twice ?*a' , [ '2' , '*' , '?a' ] ) ,
( 'sum ?*a and ?*b' , [ '?a' , '+' , '?b' ] ) ,
( 'half ?*a' , [ '?a' , '/' , '2' ] ) ,
( 'square ?*a' , [ '?a' , '*' , '?a' ] ) ,
( '?*a plus ?*b' , [ '?a' , '+' , '?b' ] ) ,
( '?*a minus ?*b' , [ '?a' , '-' , '?b' ] )
```

Replace item function()

```
def replace_item(n,o,lst):
    if lst == []:
        return []
    elif o == lst:
        return n
    elif type(lst) is not list:
        return lst
    else:
        ans = replace_item(n,o,lst[1:])
        ans.insert(0,replace_item(n,o,lst[0]))
        return ans
```

➤ Code of Translate to expression module

```
def translate_to_exp(ewlst):
    print('Input: ', ewlst)
    for pattern, response in student_rules:
        pattern = pattern.split()
        bindings = match_pattern(pattern, ewlst)
        if bindings:
            print('Rule: ', pattern, ' ', response)
            print('Binding:', bindings)
            for var, val in bindings.items():
                tval = translate_to_exp(val)
                response = replace_item(tval, '?' + var, response)
            print('Output:', response)
            return response
    if ewlst == []:
        return ewlst
    return ewlst[0]
```

➤ Simple small example Tracing

Considering the above rule, let us translate a small example question

(what is twice of square of z . if z is 3). Note that “of” will be removed as it is a noise word

```
>>> translate_to_expression ()
```

Enter Statement: what is twice of square of z . if z is 3

Input: ['what', 'is', 'twice', 'square', 'z']

Rule: ['?*a', 'is', '?*b'] ['?a', '=', '?b']

Binding: {'a': ['what'], 'b': ['twice', 'square', 'z']}

Input: ['what']

Input: ['twice', 'square', 'z']

Rule: ['twice', '?*a'] ['2', '*', '?a']

Binding: {'a': ['square', 'z']}

Input: ['square', 'z']

Rule: ['square', '?*a'] ['?a', '*', '?a']

Binding: {'a': ['z']}

Input: ['z']

Output: ['z', '*', 'z']

Output: ['2', '*', ['z', '*', 'z']]

Output: ['what', '=', ['2', '*', ['z', '*', 'z']]]

Input: ['z', 'is', '3']

Rule: ['?*a', 'is', '?*b'] ['?a', '=', '?b']

Binding: {'a': ['z'], 'b': ['3']}

Input: ['z']

Input: ['3']

Output: ['z', '=', '3']

➤ Example 2 of translate to expression

Enter Statement: x plus y is equal to z minus 5 . find square of z . 6 minus x is 4 . and 8 divided by y is 1/4

Input: ['x', 'plus', 'y', 'is', 'equal', 'to', 'z', 'minus', '5']

Rule: ['?*a', 'is', 'equal', 'to', '?*b'] ['?a', '=', '?b']

Binding: {'a': ['x', 'plus', 'y'], 'b': ['z', 'minus', '5']}

Input: ['x', 'plus', 'y']

Rule: ['?*a', 'plus', '?*b'] ['?a', '+', '?b']

Binding: {'a': ['x'], 'b': ['y']}

Input: ['x']

Input: ['y']

Output: ['x', '+', 'y']

Input: ['z', 'minus', '5']

Rule: ['?*a', 'minus', '?*b'] ['?a', '-', '?b']

Binding: {'a': ['z'], 'b': ['5']}

Input: ['z']

Input: ['5']

Output: ['z', '-', '5']

Output: [['x', '+', 'y'], '=', ['z', '-', '5']]

Input: ['find', 'square', 'z']

Rule: ['find', '?*a'] ['To-find', '=', '?a']

Binding: {'a': ['square', 'z']}

Input: ['square', 'z']

Rule: ['square', '?*a'] ['?a', '*', '?a']

Binding: {'a': ['z']}

```
Input:  ['z']
Output: ['z', '*', 'z']
Output: ['To-find', '=', ['z', '*', 'z']]
Input:  ['6', 'minus', 'x', 'is', '4']
Rule:   ['?*a', 'is', '?*b']    ['?a', '=', '?b']
Binding: {'a': ['6', 'minus', 'x'], 'b': ['4']}
Input:  ['6', 'minus', 'x']
Rule:   ['?*a', 'minus', '?*b']  ['?a', '-', '?b']
Binding: {'a': ['6'], 'b': ['x']}
Input:  ['6']
Input:  ['x']
Output: ['6', '-', 'x']
Input:  ['4']
Output: [['6', '-', 'x'], '=', '4']
Input:  ['8', 'divided', 'by', 'y', 'is', '1/4']
Rule:   ['?*a', 'is', '?*b']    ['?a', '=', '?b']
Binding: {'a': ['8', 'divided', 'by', 'y'], 'b': ['1/4']}
Input:  ['8', 'divided', 'by', 'y']
Rule:   ['?*a', 'divided', 'by', '?*b']  ['?a', '/', '?b']
Binding: {'a': ['8'], 'b': ['y']}
Input:  ['8']
Input:  ['y']
Output: ['8', '/', 'y']
Input:  ['1/4']
Output: [['8', '/', 'y'], '=', '1/4']
```

STUDENT CODE UPTO TRANSLATE TO EXPRESSION MODULE

```

from patmatch import * # or write the patmatch code in this file
operators = "+-*/%"
numbers = "0123456789"
letters = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ"
student_rules = [
    ('find ?*a' , ['To-find' , '=', '?a']),
    ('?*a is equal to ?*b' , ['?a' , '=', '?b']),
    ('?*a makes ?*b' , ['?a' , '=', '?b']),
    ('?*a are calculated by ?*b' , ['?a' , '=', '?b']),
    ('?*a calculated by ?*b' , ['?a' , '=', '?b']),
    ('?*a can be calculated as ?*b' , ['?a' , '=', '?b']),
    ('?*a calculated as ?*b' , ['?a' , '=', '?b']),
    ('?*a can be ?*b' , ['?a' , '=', '?b']),
    ('?*a equal to ?*b' , ['?a' , '=', '?b']),
    ('?*a equals ?*b' , ['?a' , '=', '?b']),
    ('?*a is ?*b' , ['?a' , '=', '?b']),
    ('?*a are ?*b' , ['?a' , '=', '?b']),
    ('?*a in ?*b' , ['?a' , '=', '?b']),
    ('?*a plus ?*b' , ['?a' , '+', '?b']),
    ('?*a minus ?*b' , ['?a' , '-', '?b']),
    ('?*a divided by ?*b' , ['?a' , '/', '?b']),
    ('twice ?*a' , ['2' , '*', '?a']),
    ('sum ?*a and ?*b' , ['?a' , '+', '?b']),
    ('adding ?*a and ?*b' , ['?a' , '+', '?b']),
    ('distributed into ?*a and ?*b' , ['?a' , '+', '?b']),
    ('divided into ?*a and ?*b' , ['?a' , '+', '?b']),
    ('one half ?*a' , ['?a' , '/', '2']),
    ('half ?*a' , ['?a' , '/', '2']),
    ('square ?*a' , ['?a' , '*', '?a']),
    ('?*a times ?*b' , ['?a' , '*', '?b']),
    ('?*a multiplied by ?*b' , ['?a' , '*', '?b']),
    ('?*a multiplied with ?*b' , ['?a' , '*', '?b']),
    ('difference between ?*a and ?*b' , ['?b' , '-', '?a']),
    ('?*a % less than ?*b' , ['?b' , '/', ['1' , '-', ['?a' , '/', '100']]])
]

def toeqstr(elst):
    ans = ''
    if type(elst) is str:
        return elst
    for item in elst:
        if type(item) is list:
            ans = ans + " ( " + toeqstr(item) + " )"
        else:
            if ans == '':
                ans = item
            else:
                ans = ans + ' ' + str(item)
    return ans

def replace_item(n,o,lst):
    if lst == []:
        return []
    elif o == lst:
        return n
    elif type(lst) is not list:
        return lst
    else:
        ans = replace_item(n,o,lst[1:])
        ans.insert(0,replace_item(n,o,lst[0]))
        return ans

```

```
def translate_to_exp(eqwlst):
    print('Input: ', toeqstr(eqwlst), '')
    for pattern, response in student_rules:
        pattern = pattern.split()
        bindings = match_pattern(pattern, eqwlst)
        if bindings:
            print('Rule:', toeqstr(pattern), ' ', [' ', toeqstr(response), ''])
            print('Binding:', bindings)
            for var, val in bindings.items():
                #print(var)
                tval = translate_to_exp(val)
                response = replace_item(tval, '?' + var, response)
            print('Output:', response)
            return response
    if eqwlst == []:
        return eqwlst
    return eqwlst[0]

def remove_noise(txt):
    nwords = ['a', 'the', 'number', 'of', 'this', 'if', 'that',
              '$', 'his', 'while']
    for w in nwords:
        if txt.count(w) > 0:
            txt = [a for a in txt if a != w]
    return txt

def student():
    userinput = input("Enter Statement:")
    userinput = userinput.lower()
    engEqs = userinput.split('.')
    eqs = []
    for engeq in engEqs:
        engeq = engeq.split()
        eqwlst = remove_noise(engeq)
        eq1 = translate_to_exp(eqwlst)
        eqs.append(eq1)
    eqs = [eq for eq in eqs if eq != []]
    solve_equations(eqs)

def solve_equations(eqs):
    print("UNSOLVED EQUATIONS")
    print_equations(eqs)
    print('SOLVED EQUATIONS')
    #solved = solve(eqs)          # solve module in-active
    #print_equations(solved)

def print_equations(eqs):
    for eq in eqs:
        print(toeqstr(eq))
```